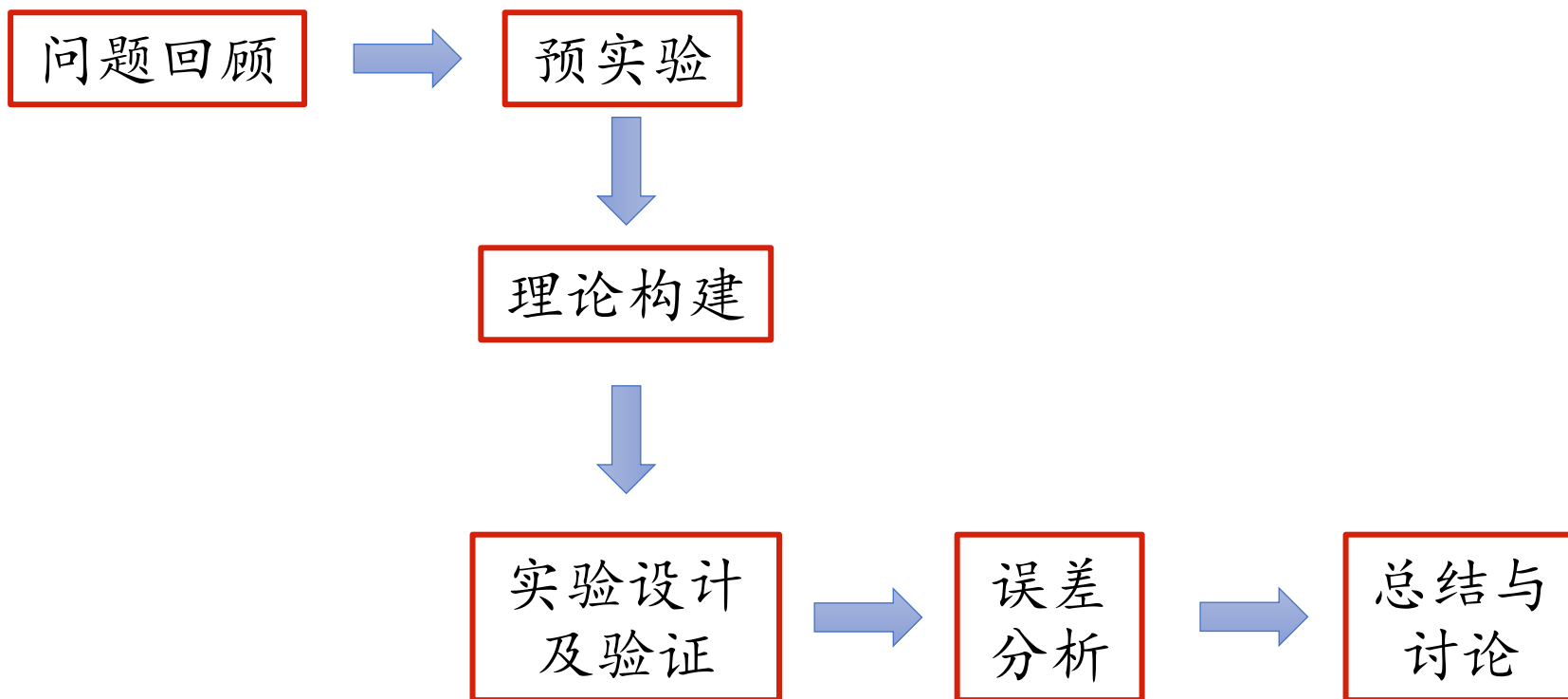




四川大學
SICHUAN UNIVERSITY

气垫弹射器

四川大学代表队



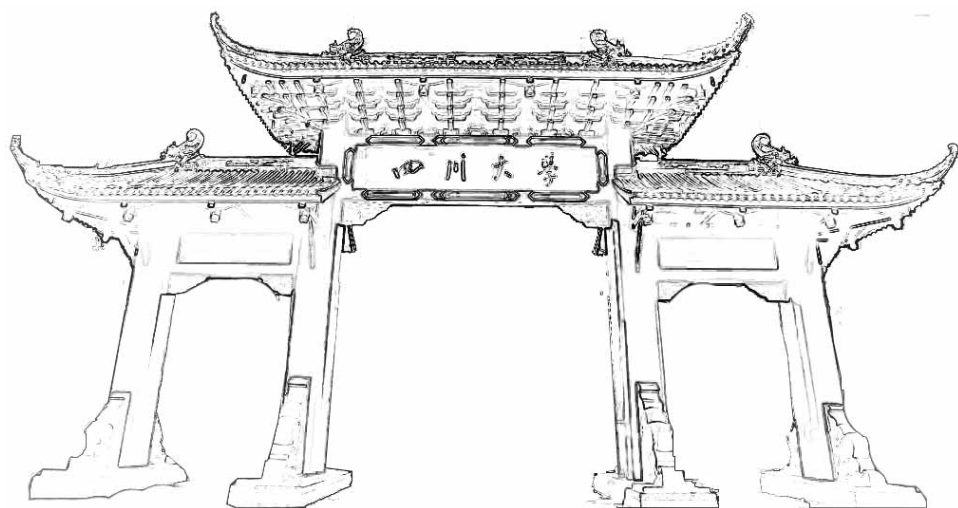


Place an object on a large air cushion and drop several other objects in such a way that the first object is catapulted away. Investigate how the exit velocity depends on relevant parameters.

将一个物体放在一个大气垫上，然后投放其他几个物体，使第一个物体被弹射出去。研究弹出速度如何取决于相关参数。



- 1、研究内容（contents）：弹出速度如何取决于相关参数
(How the exit velocity depends on relevant parameters)
- 2、研究对象：气垫和物体（入射物和出射物）
(Air cushion and objects)
- 3、问题关键：
 - ①气垫：大气垫（Large air cushion）
 - ②出射物体：一个物体（An object）
 - ③要求：投放其他几个物体（drop several other objects）
 - ④方式：弹射出去（Catapulted away）



预实验



从预实验可以看出：

- 1、初步判断出运动过程：物体掉落至气球上引起气球膜形变、气球内气体流动、气球膜进一步形变、另一个物体被弹起。
- 2、能量方面：入射气体动能主要转化为出射物体动能，也以声能、其体内能、气球膜弹性势能、气球动能消耗。



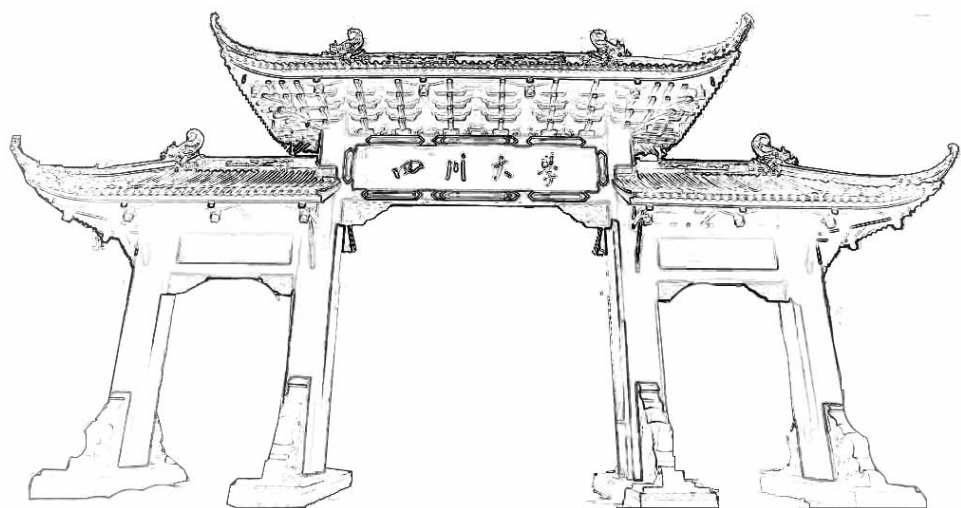
同时，进行多次实验时我们发现如下规律：

- 1、入射速度越大，弹出速度越大。
- 2、入射物体质量越大，弹出速度越大。
- 3、弹出速度也受入射位置、入射物体数量影响



由预实验，提出以下相关参数：

- 1、入射物体速度
- 2、入射物体个数
- 3、入射物体质量
- 4、出射物体质量
- 5、入射位置



理论构建



由预实验和分析结果，我们对模型作出以下理想简化：

- 1、将气体视为不可压缩牛顿流体。
- 2、认为在碰撞后入射物体动能完全转化出射物体动能和为其他形式能量。
- 3、忽略气垫动能。
- 4、忽略空气阻力造成的能量损失。
- 5、气垫位移可以忽略。
- 6、气垫膜弹性势能可以忽略



一、气体

将气垫视作由闭合线弹性膜包围一定量不可压缩气体组成的曲面。在运动时，对每个气体微元（粒子），其每个方向上的受力为两部分，粘滞力和压力。以x方向为例

$$F_x = F_f + F_p$$

其中，由牛顿粘性定律：

$$F_f = \mu \left(d \frac{\partial u}{\partial y} dx dz + d \frac{\partial u}{\partial z} dx dy \right)$$

并且 F_p 可表示为：

$$F_p = dp dy dz$$



同理，对于y、z方向：

$$F_x = \mu \left(d \frac{\partial u}{\partial y} dx dz + d \frac{\partial u}{\partial z} dx dy \right) + dp_x dy dz$$

$$F_y = \mu \left(d \frac{\partial v}{\partial x} dy dz + d \frac{\partial v}{\partial z} dy dz \right) - \rho dx dy dz \vec{g} + dp_y dx dz$$

$$F_z = \mu \left(d \frac{\partial w}{\partial x} dy dz + d \frac{\partial w}{\partial y} dx dz \right) + dp_z dx dy$$

对于每个流体微元，质量为 $\rho dx dy dz$ ，由牛顿第二定律：

$$\rho \frac{\partial u}{\partial t} = \mu \left(\frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2} \right) + \frac{dp}{dx}$$

$$\rho \frac{\partial v}{\partial t} = \mu \left(\frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 v}{\partial z^2} \right) + \frac{dp}{dy} - \rho \vec{g}$$

$$\rho \frac{\partial w}{\partial t} = \mu \left(\frac{\partial^2 w}{\partial x^2} + \frac{\partial^2 w}{\partial y^2} \right) + \frac{dp}{dz}$$



由于假定流体不可压缩，可知 $\nabla \vec{V} = 0$ 。因此方程可写作

$$\rho \frac{\partial \vec{V}}{\partial t} = \mu \nabla^2 \vec{V} - \rho \vec{g} - \nabla p$$

二、气垫

我们使用的气垫材料为铝膜，在小形变下可视作线弹性材料。其本构关系为：

$$\sigma = E\epsilon$$

对于膜上一点，用液体膜表面张力近似内部应力，根据Young-Laplace Equation:

$$\Delta p = p - p_0 = \gamma_{eq} \left(\frac{1}{R_1} + \frac{1}{R_2} \right)$$

其中， γ_{eq} 是等效表面张力系数， R_1, R_2 是主曲率半径。



我们可以将等效表面张力系数表示为：

$$\gamma_{eq} = \frac{Eh}{3(1-\nu^2)}$$

故对于边界层，其压强差可表示为：

$$\Delta p = \frac{Eh}{3(1-\nu^2)} \left(\frac{1}{R_1} + \frac{1}{R_2} \right)$$

因此，问题可表示为在冲击与载荷下系统：

$$\rho \frac{\partial \vec{V}}{\partial t} = \mu \nabla^2 \vec{V} - \rho \vec{g} - \nabla p$$

$$p_b = p_0 + \frac{Eh}{3(1-\nu^2)} \left(\frac{1}{R_1} + \frac{1}{R_2} \right)$$

的动力学响应。

三、数值模拟

基于先前的分析，我们采用平滑粒子流体动力学（SPH）方法对该系统进行数值模拟。

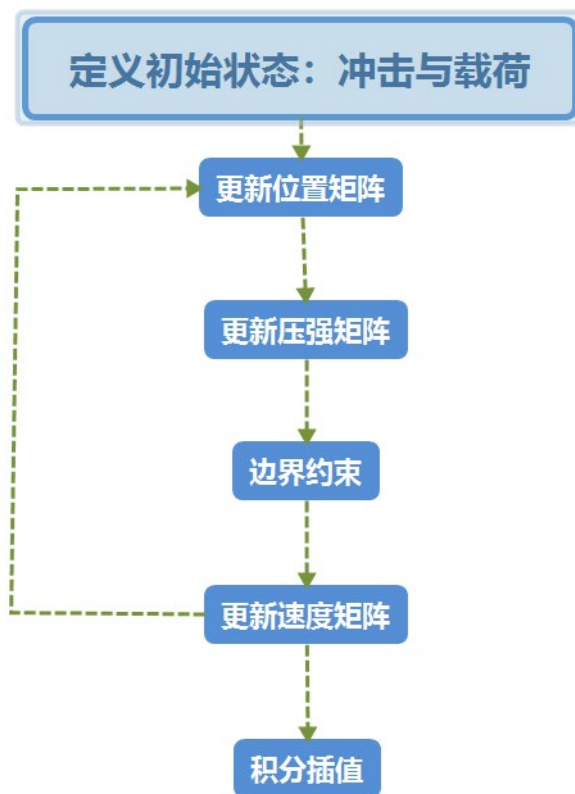
考虑将气体离散为一系列粒子，对于索引（i, j, k）的粒子，其受力为：

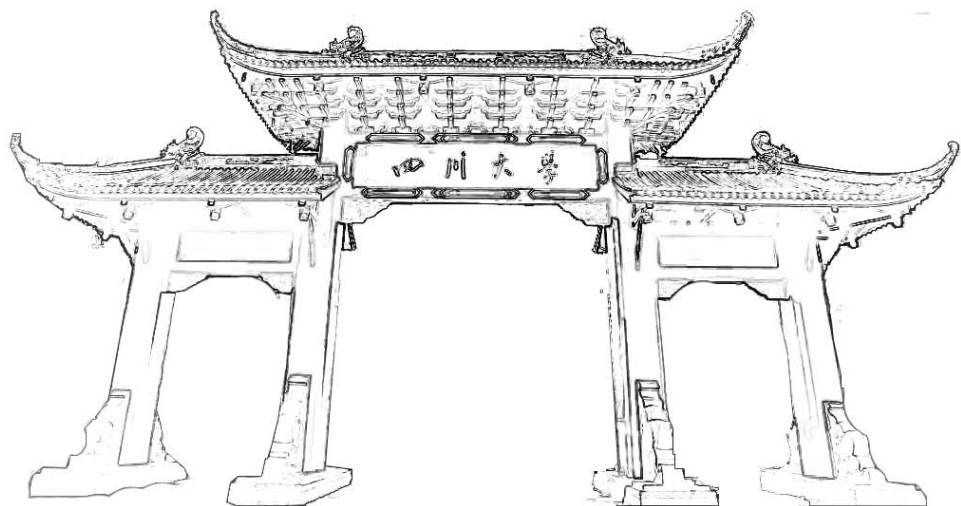
$$F_{ijk} = \begin{bmatrix} (P_{i-1jk} - P_{i+1jk}) \lambda_2 \lambda_3 / \rho^2 \left[\mu \left(\frac{dY_{i+1jk}}{dt} + \frac{dY_{i-1jk}}{dt} - 2 \frac{dY_{ijk}}{dt} \right) \lambda_2 \lambda_3 / \rho^2 \right. \\ \left. \mu \left(\frac{dZ_{i+1k}}{dt} + \frac{dZ_{i-1k}}{dt} - 2 \frac{dZ_{ijk}}{dt} \right) \lambda_2 \lambda_3 / \rho^2 \right] \\ \mu \left[\frac{dX_{ij+1k}}{dt} + \frac{dX_{ij-1k}}{dt} - 2 \frac{dX_{ijk}}{dt} \right] \lambda_1 \lambda_3 / \rho^2 (P_{ij+1k} - P_{ij-1k}) \lambda_1 \lambda_3 / \rho^2 \left[\mu \left(\frac{dZ_{ij+1k}}{dt} + \frac{dZ_{ij-1k}}{dt} - 2 \frac{dZ_{ijk}}{dt} \right) \lambda_1 \lambda_3 / \rho^2 \right. \\ \left. \mu \left(\frac{dX_{ijk+1}}{dt} + \frac{dX_{ijk-1}}{dt} - 2 \frac{dX_{ijk}}{dt} \right) \lambda_1 \lambda_2 / \rho^2 (P_{ijk+1} - P_{ijk-1}) \lambda_1 \lambda_2 / \rho^2 \right] \end{bmatrix}$$

由于忽略切向力和边界层滑移，边界粒子受力方向相邻坐标的主方向。



以下是我们的程序计算框架：





实验设计及验证



- 1、实验原理及目的：以不同的参数设置进行多组实验，测量出射速度与数值模拟结果对比验证。
- 2、实验器材：橡皮、气球、刻度尺
- 3、实验自变量：入射物体个数，入射位置，入射物体质量、入射速度
- 4、实验因变量：出射速度
- 5、控制变量实验：
 - ①只改变橡皮个数
 - ②只改变入射橡皮质量
 - ③只改变出射橡皮质量
 - ④只改变入射速度
 - ⑤只改变入射位置



正式实验

实验一：只改变入射位置，其他量保持不变。

出射速度 (m/s) 入射位置 (cm)	实验	理论
5.26	4.27	5.13
9.95	1.94	2.11

其他条件：入射速度3.02m/s，入射一个物体质量12.72g，出射橡皮质量6.36g



正式实验

实验二：只改变橡皮个数，其他量保持不变。

出射速度 (m/s) 橡皮个数 (个)	实验	理论
1	0.84	1.12
2	1.30	2.14
3	1.97	2.20
4	2.86	3.23
5	3.52	3.43

其他条件：入射速度 $1.82 \pm 0.16 \text{ m/s}$ ，入射物体总质量 19.08 g ，出射橡皮质量 6.36 g



正式实验

实验三：只改变出射橡皮质量，其他量保持不变。

出射速度 (m/s) 出射橡皮质量 (g)	实验	理论
6.36	1.21	1.12
12.72	1.84	1.93

其他条件：入射速度 $3.34 \pm 0.04 \text{ m/s}$ ，入射一个物体质量 6.36 g ，入射位置 $9.55 \pm 0.13 \text{ cm}$



正式实验

实验四：只改变入射橡皮质量，其他量保持不变。

出射速度 (m/s) 入射橡皮质量 (g)	实验	理论
6.36	1.21	1.99
12.72	2.84	2.58

其他条件：入射速度 $3.20 \pm 0.18 \text{m/s}$ ，入射一个物体，入射位置 $5.44 \pm 0.18 \text{cm}$ ，出射橡皮质量 6.36g

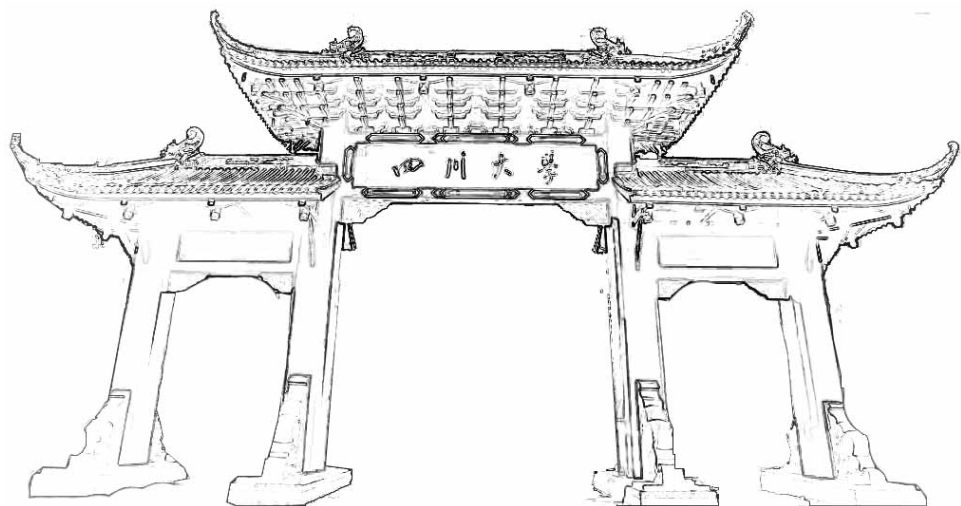


正式实验

实验五：只改变入射速度，其他量保持不变。

出射速度 (m/s) 入射速度 (m/s)	实验	理论
2.00	1.19	1.93
2.13	1.21	1.94
2.71	1.47	1.52
3.42	1.96	2.31
3.71	2.11	2.11

其他条件：入射质量6.36g，入射一个物体，入射位置 $5.44 \pm 0.18\text{cm}$ ，出射橡皮质量6.36g



误差分析



实验误差

- 1、**基本参量的测量**上由于仪器设备限制导致测量结果的误差
- 2、在处理边界层形状时，我们近似为两个球冠，而实际上是椭球冠。根据实际实验，曲率计算误差造成的边界层压强误差在22.7%
- 3、气垫实际由两层膜缝合制成，缝合处会对膜内应力分布产生难以评估的影响。
- 4、用手释放橡皮导致接触气垫时橡皮并非“平躺”，进而出现翻滚，可能导致一系列复杂的影响。



- [1] 梁昆淼，力学【下册】. 高等教育出版社 2009.7
- [2] A. Chorin, “Numerical solution of the navier–stokes equations,” Mathematics of Computation, vol. 22, 10 1968.
- [3] 赵亚博，表面与界面物理力学. 北京：科学出版社，2012.

感谢聆听！

附录



```

mu = 1.48*10^-5; rho = 1.17; module = 7*10^10;
% Dynamic viscosity, density of air, Young's module of membrane.
dl = 0.01; dm = rho*dl^3; Gm0 = 6.36 * 10^-3 *9.8;
% Distance between particle, mass of particle, gravity of a particle.
nx = 30; ny = 30; nz = 12; sz = [nx, ny, nz];
% Number of particle on each direction.
time = 25; dt = 0.01; % total run time.
velocity_inject = 3.2;
% Injecting velocity, assuming reach a common velocity instantly.
sigma = dl;

isBoundary = zeros(sz(1), sz(2), sz(3));
isBoundary(1, :, :) = 1; isBoundary(nx, :, :) = 1;
isBoundary(:, 1, :) = 1; isBoundary(:, ny, :) = 1;
isBoundary(:, :, 1) = 1; isBoundary(:, :, nz) = 1;
isBoundary = logical(isBoundary);
% Indexes of boundary particles.

hit_zone = zeros(sz(1), sz(2), sz(3));
hit_zone(9:11, 14:16, nz) = 1;
hit_zone = logical(hit_zone);
load_zone = zeros(sz(1), sz(2), sz(3));
load_zone(19:21, 14:16, :) = 1;
load_zone = logical(load_zone);
% Set hit zone and loaded zone.

x = zeros(nx, ny, nz, time);
y = zeros(nx, ny, nz, time);
z = zeros(nx, ny, nz, time);
u = zeros(nx, ny, nz, time);
v = zeros(nx, ny, nz, time);
w = zeros(nx, ny, nz, time);
p = 101325*ones(nx, ny, nz, time);

d2u_dx2 = zeros(sz(1), sz(2), sz(3)-1);
d2v_dx2 = zeros(sz(1), sz(2), sz(3)-1);
d2w_dx2 = zeros(sz(1), sz(2), sz(3)-1);
d2u_dy2 = zeros(sz(1), sz(2), sz(3)-1);
d2v_dy2 = zeros(sz(1), sz(2), sz(3)-1);
d2w_dy2 = zeros(sz(1), sz(2), sz(3)-1);
d2u_dz2 = zeros(sz(1), sz(2), sz(3)-1);
d2v_dz2 = zeros(sz(1), sz(2), sz(3)-1);
d2w_dz2 = zeros(sz(1), sz(2), sz(3)-1);
du_dx = zeros(sz(1), sz(2), sz(3)-1);

```

```

dv_dx = zeros(sz(1), sz(2), sz(3)-1);
dw_dx = zeros(sz(1), sz(2), sz(3)-1);
du_dy = zeros(sz(1), sz(2), sz(3)-1);
dv_dy = zeros(sz(1), sz(2), sz(3)-1);
dw_dy = zeros(sz(1), sz(2), sz(3)-1);
du_dz = zeros(sz(1), sz(2), sz(3)-1);
dv_dz = zeros(sz(1), sz(2), sz(3)-1);
dw_dz = zeros(sz(1), sz(2), sz(3)-1);
dp_dx = zeros(sz(1), sz(2), sz(3)-1);
dp_dy = zeros(sz(1), sz(2), sz(3)-1);
dp_dz = zeros(sz(1), sz(2), sz(3)-1);

fx = zeros(sz(1), sz(2), sz(3));
fy = zeros(sz(1), sz(2), sz(3));
fz = zeros(sz(1), sz(2), sz(3));

for ii=1:nx
    x(ii, :, :, 1) = ii * dl;
end

for ii=1:ny
    y(:, ii, :, 1) = ii * dl;
end

for ii=1:nz
    z(:, :, ii, 1) = ii * dl;
end

w_slice = w(:, :, :, 1);
w_slice(hit_zone) = velocity_inject;
w(:, :, :, 1) = w_slice;
% Initialize matrixes.

for t=2:time-1 % Operation for each time step
    x(:, :, :, t) = x(:, :, :, t-1) + u(:, :, :, t-1)*dt;
    y(:, :, :, t) = y(:, :, :, t-1) + v(:, :, :, t-1)*dt;
    z(:, :, :, t) = z(:, :, :, t-1) + w(:, :, :, t-1)*dt;
    % Update locations due to the velocity matrixes on last time step.

    for j=2:sz(2)-1

```

```

        for j=2:sz(2)-1
            for k=2:sz(3)-1
                dx = (x(i+1, j, k, t) + x(i-1, j, k, t) - 2*x(i, j, k, t))/2;
                dy = (y(i, j+1, k, t) + y(i, j-1, k, t) - 2*y(i, j, k, t))/2;
                dz = (z(i, j, k+1, t) + z(i, j, k-1, t) - 2*z(i, j, k, t))/2;

                d2u_dx2(i, j, k) = (u(i-1, j, k, t)+u(i+1, j, k, t)-2*u(i, j, k, t))/(dx^2);
                d2u_dy2(i, j, k) = (u(i, j-1, k, t)+u(i, j+1, k, t)-2*u(i, j, k, t))/(dy^2);
                d2u_dz2(i, j, k) = (u(i, j, k-1, t)+u(i, j, k+1, t)-2*u(i, j, k, t))/(dz^2);

                du_dx(i, j, k) = (u(i+1, j, k, t)-u(i-1, j, k, t))/(2*dx);
                du_dy(i, j, k) = (u(i, j+1, k, t)-u(i, j-1, k, t))/(2*dy);
                du_dz(i, j, k) = (u(i, j, k+1, t)-u(i, j, k-1, t))/(2*dz);

                d2v_dx2(i, j, k) = (v(i-1, j, k, t)+v(i+1, j, k, t)-2*v(i, j, k, t))/(dx^2);
                d2v_dy2(i, j, k) = (v(i, j-1, k, t)+v(i, j+1, k, t)-2*v(i, j, k, t))/(dy^2);
                d2v_dz2(i, j, k) = (v(i, j, k-1, t)+v(i, j, k+1, t)-2*v(i, j, k, t))/(dz^2);

                dv_dx(i, j, k) = (v(i+1, j, k, t)-v(i-1, j, k, t))/(2*dx);
                dv_dy(i, j, k) = (v(i, j+1, k, t)-v(i, j-1, k, t))/(2*dy);
                dv_dz(i, j, k) = (v(i, j, k+1, t)-v(i, j, k-1, t))/(2*dz);

                d2w_dx2(i, j, k) = (w(i-1, j, k, t)+w(i+1, j, k, t)-2*w(i, j, k, t))/(dx^2);
                d2w_dy2(i, j, k) = (w(i, j-1, k, t)+w(i, j+1, k, t)-2*w(i, j, k, t))/(dy^2);
                d2w_dz2(i, j, k) = (w(i, j, k-1, t)+w(i, j, k+1, t)-2*w(i, j, k, t))/(dz^2);

                dw_dx(i, j, k) = (w(i+1, j, k, t)-w(i-1, j, k, t))/(2*dx);
                dw_dy(i, j, k) = (w(i, j+1, k, t)-w(i, j-1, k, t))/(2*dy);
                dw_dz(i, j, k) = (w(i, j, k+1, t)-w(i, j, k-1, t))/(2*dz);

                du_dt(i, j, k) = (u(i, j, k, t+1)-u(i, j, k, t-1))/(2*dt);
                dv_dt(i, j, k) = (v(i, j, k, t+1)-v(i, j, k, t-1))/(2*dt);
                dw_dt(i, j, k) = (w(i, j, k, t+1)-w(i, j, k, t-1))/(2*dt);

            end
        end
    end

```

附录



```

end
dp_dx = mu*(d2u_dx2 + d2u_dy2 + d2u_dz2)-rho*(du_dt + u(:, :, 1:sz(3)-1, t).*du_dx + u(:, :, 1:sz(3)-1, t).*du_dy + u(:, :, 1:sz(3)-1, t).*du_dz);
dp_dy = mu*(d2v_dx2 + d2v_dy2 + d2v_dz2)-rho*(dv_dt + v(:, :, 1:sz(3)-1, t).*dv_dx + v(:, :, 1:sz(3)-1, t).*dv_dy + v(:, :, 1:sz(3)-1, t).*dv_dz);
dp_dz = mu*(d2w_dx2 + d2w_dy2 + d2w_dz2)-rho*(dw_dt + w(:, :, 1:sz(3)-1, t).*dw_dx + w(:, :, 1:sz(3)-1, t).*dw_dy + w(:, :, 1:sz(3)-1, t).*dw_dz);
% Solve Naiver-Stokes Equation by finite difference.
dp_x = dp_dx*dx;
dp_y = dp_dy*dy;
dp_z = dp_dz*dz;
for i=2:sz(1)-1
    for j=2:sz(2)-1
        for k=2:sz(3)-1
            p_ave = [p(i-1, j, k, t), p(i, j-1, k, t), p(i, j, k-1, t)];
            p(i, j, k, t) = sum(p_ave + [dp_x(i, j, k), dp_y(i, j, k), dp_z(i, j, k)])/3;
        end
    end
end
for iteration=1:600
    for i=2:sz(1)-1
        for j=2:sz(2)-1
            for k=2:sz(3)-1
                p_ave = [p(i-1, j, k, t), p(i, j-1, k, t), p(i, j, k-1, t)];
                p(i, j, k, t) = p(i, j, k, t)/2 + sum(p_ave + [dp_x(i, j, k), dp_y(i, j, k), dp_z(i, j, k)])/6;
            end
        end
    end
end
% Compute pressure for each particle due to the velocity matrixies on
% last time step. Iterating for 600 times to ensure convergency.
for i=2:sz(1)-1

```

```

    for j=2:sz(2)-1
        for k=2:sz(3)-1
            dp_x = p(i+1, j, k, t) - p(i-1, j, k, t);
            dp_y = p(i, j+1, k, t) - p(i, j-1, k, t);
            dp_z = p(i, j, k+1, t) - p(i, j, k-1, t);
            f_x(i, j, k) = -4*mu*d1^2 * (du_dy(i, j, k) + dv_dx(i, j, k) + dp_x);
            f_y(i, j, k) = -4*mu*d1^2 * (dv_dy(i, j, k) + dw_dx(i, j, k) + dp_y);
            f_z(i, j, k) = -4*mu*d1^2 * (dw_dy(i, j, k) + dp_z);
        end
    end
end
% Compute forces on internal particles.
% Newton's law of internal friction and pressure difernce.
for jj=2:nx-1
    for kk=2:nz-1
        vec1 = [x(1, jj-1, kk)-x(1, jj, kk), y(1, jj-1, kk)-y(1, jj, kk), z(1, jj-1, kk)-z(1, jj, kk)];
        vec2 = [x(1, jj+1, kk)-x(1, jj, kk), y(1, jj+1, kk)-y(1, jj, kk), z(1, jj+1, kk)-z(1, jj, kk)];
        vec3 = [x(1, jj, kk-1)-x(1, jj, kk), y(1, jj, kk-1)-y(1, jj, kk), z(1, jj, kk-1)-z(1, jj, kk)];
        vec4 = [x(1, jj, kk+1)-x(1, jj, kk), y(1, jj, kk+1)-y(1, jj, kk), z(1, jj, kk+1)-z(1, jj, kk)];
        T = module*((norm(vec1)-d1)*vec1/norm(vec1)+(norm(vec2)-d1)*vec2/norm(vec2)+(norm(vec3)-d1)*vec3/norm(vec3)+(norm(vec4)-d1)*vec4/norm(vec4));
        f_x(1, jj, kk) = f_x(1, jj, kk) + T(1); f_y(1, jj, kk) = f_y(1, jj, kk) + T(2); f_z(1, jj, kk) = f_z(1, jj, kk) + T(3);
        vec1 = [x(nx, jj-1, kk)-x(nx, jj, kk), y(nx, jj-1, kk)-y(nx, jj, kk), z(nx, jj-1, kk)-z(nx, jj, kk)];
        vec2 = [x(nx, jj+1, kk)-x(nx, jj, kk), y(nx, jj+1, kk)-y(nx, jj, kk), z(nx, jj+1, kk)-z(nx, jj, kk)];
        vec3 = [x(nx, jj, kk-1)-x(nx, jj, kk), y(nx, jj, kk-1)-y(nx, jj, kk), z(nx, jj, kk-1)-z(nx, jj, kk)];
        vec4 = [x(nx, jj, kk+1)-x(nx, jj, kk), y(nx, jj, kk+1)-y(nx, jj, kk), z(nx, jj, kk+1)-z(nx, jj, kk)];
        T = module*((norm(vec1)-d1)*vec1/norm(vec1)+(norm(vec2)-d1)*vec2/norm(vec2)+(norm(vec3)-d1)*vec3/norm(vec3)+(norm(vec4)-d1)*vec4/norm(vec4));
        f_x(1, jj, kk) = f_x(1, jj, kk) + T(1); f_y(1, jj, kk) = f_y(1, jj, kk) + T(2); f_z(1, jj, kk) = f_z(1, jj, kk) + T(3);
        vec1 = [x(nx, jj-1, kk)-x(nx, jj, kk), y(nx, jj-1, kk)-y(nx, jj, kk), z(nx, jj-1, kk)-z(nx, jj, kk)];
        vec2 = [x(nx, jj+1, kk)-x(nx, jj, kk), y(nx, jj+1, kk)-y(nx, jj, kk), z(nx, jj+1, kk)-z(nx, jj, kk)];
        vec3 = [x(nx, jj, kk-1)-x(nx, jj, kk), y(nx, jj, kk-1)-y(nx, jj, kk), z(nx, jj, kk-1)-z(nx, jj, kk)];
        vec4 = [x(nx, jj, kk+1)-x(nx, jj, kk), y(nx, jj, kk+1)-y(nx, jj, kk), z(nx, jj, kk+1)-z(nx, jj, kk)];
        T = module*((norm(vec1)-d1)*vec1/norm(vec1)+(norm(vec2)-d1)*vec2/norm(vec2)+(norm(vec3)-d1)*vec3/norm(vec3)+(norm(vec4)-d1)*vec4/norm(vec4));
        f_x(1, jj, kk) = f_x(1, jj, kk) + T(1); f_y(1, jj, kk) = f_y(1, jj, kk) + T(2); f_z(1, jj, kk) = f_z(1, jj, kk) + T(3);
    end
end

```

```

        T = module*((norm(vec1)-d1)*vec1/norm(vec1)+(norm(vec2)-d1)*vec2/norm(vec2)+(norm(vec3)-d1)*vec3/norm(vec3)+(norm(vec4)-d1)*vec4/norm(vec4));
        f_x(nx, jj, kk) = f_x(nx, jj, kk) - T(1); f_y(nx, jj, kk) = f_y(nx, jj, kk) - T(2); f_z(nx, jj, kk) = f_z(nx, jj, kk) - T(3);
    end
end
for ii=2:nx-1
    for kk=2:nz-1
        vec1 = [x(ii-1, 1, kk)-x(ii, 1, kk), y(ii-1, 1, kk)-y(ii, 1, kk), z(ii-1, 1, kk)-z(ii, 1, kk)];
        vec2 = [x(ii+1, 1, kk)-x(ii, 1, kk), y(ii+1, 1, kk)-y(ii, 1, kk), z(ii+1, 1, kk)-z(ii, 1, kk)];
        vec3 = [x(ii, 1, kk-1)-x(ii, 1, kk), y(ii, 1, kk-1)-y(ii, 1, kk), z(ii, 1, kk-1)-z(ii, 1, kk)];
        vec4 = [x(ii, 1, kk+1)-x(ii, 1, kk), y(ii, 1, kk+1)-y(ii, 1, kk), z(ii, 1, kk+1)-z(ii, 1, kk)];
        T = module*((norm(vec1)-d1)*vec1/norm(vec1)+(norm(vec2)-d1)*vec2/norm(vec2)+(norm(vec3)-d1)*vec3/norm(vec3)+(norm(vec4)-d1)*vec4/norm(vec4));
        f_x(ii, 1, kk) = f_x(ii, 1, kk) + T(1); f_y(ii, 1, kk) = f_y(ii, 1, kk) + T(2); f_z(ii, 1, kk) = f_z(ii, 1, kk) + T(3);
        vec1 = [x(ii-1, ny, kk)-x(ii, ny, kk), y(ii-1, 1, kk)-y(ii, 1, kk), z(ii-1, 1, kk)-z(ii, 1, kk)];
        vec2 = [x(ii+1, ny, kk)-x(ii, ny, kk), y(ii+1, 1, kk)-y(ii, 1, kk), z(ii+1, 1, kk)-z(ii, 1, kk)];
        vec3 = [x(ii, ny, kk-1)-x(ii, ny, kk), y(ii, 1, kk-1)-y(ii, 1, kk), z(ii, 1, kk-1)-z(ii, 1, kk)];
        vec4 = [x(ii, ny, kk+1)-x(ii, ny, kk), y(ii, 1, kk+1)-y(ii, 1, kk), z(ii, 1, kk+1)-z(ii, 1, kk)];
        T = module*((norm(vec1)-d1)*vec1/norm(vec1)+(norm(vec2)-d1)*vec2/norm(vec2)+(norm(vec3)-d1)*vec3/norm(vec3)+(norm(vec4)-d1)*vec4/norm(vec4));
        f_x(ii, ny, kk) = f_x(ii, ny, kk) - T(1); f_y(ii, ny, kk) = f_y(ii, ny, kk) - T(2); f_z(ii, ny, kk) = f_z(ii, ny, kk) - T(3);
    end
end
for ii=2:nx-1

```

附录



```

for jj=2:nz-142
    vec1 = [x(ii-1, jj, nz)-x(ii, 1, nz), y(ii-1, jj, nz)-y(ii, jj,
nz), z(ii-1, jj, nz)-z(ii, jj, kk)];42
    vec2 = [x(ii+1, jj, nz)-x(ii, jj, nz), y(ii+1, jj, nz)-y(ii, jj,
nz), z(ii+1, jj, nz)-z(ii, jj, nz)];42
    vec3 = [x(ii, jj-1, nz)-x(ii, jj, nz), y(ii, jj-1, nz)-y(ii, jj,
nz), z(ii, jj-1, nz)-z(ii, jj, nz)];42
    vec4 = [x(ii, jj+1, nz)-x(ii, jj, nz), y(ii, jj+1, nz)-y(ii, jj,
nz), z(ii, jj+1, nz)-z(ii, jj, nz)];42

    T = module*((norm(vec1)-dl)*vec1/norm(vec1)+(norm(vec2)-
dl)*vec2/norm(vec2)+(norm(vec3)-dl)*vec3/norm(vec3)+(norm(vec4)-
dl)*vec4/norm(vec4));42

    f_x(ii, jj, nz) = f_x(ii, jj, nz) - T(1); f_y(ii, jj, nz) =
f_y(ii, jj, nz) - T(2); f_z(ii, jj, nz) = f_z(ii, jj, nz) - T(3);42
end42

% Assuming the membrane is made of linear elastical materials.42
% Use Hooke's Law solve boundary condition.42

42
42
f_z(hit_zone) = f_z(hit_zone) - Gm0;42
w_now = w(i,: , : , t);42
w_slice = w(i,: , : , t-1);42

if max(w_slice(load_zone))<max(w_now(load_zone))42
    f_z(load_zone) = f_z(load_zone) - Gm0;42
end42

42
w(i,: , 1, t) = 0;42
% Considering weight of objects.42

42
u(i,: , : , t+1) = u(: , : , t) + f_x*dt/dm;42
v(i,: , : , t+1) = v(: , : , t) + f_y*dt/dm;42
w(i,: , : , t+1) = w(: , : , t) + f_z*dt/dm;42
% Update the velocity matrices.42
end42

42
% Using Gaussian filters to smooth velocity and pressure.42
for t = 2:time42
    for i=2:sz(1)-142
        for j=2:sz(2)-142

```

```

for k=2:sz(3)-142
    point0 = [x(i, j, k, t), y(i, j, k, t), z(i, j, k, t), u(i,
j, k, t), v(i, j, k, t), w(i, j, k, t), p(i, j, k, t)];42
    point1 = [x(i+1, j, k, t), y(i+1, j, k, t), z(i+1, j, k, t),
u(i+1, j, k, t), v(i+1, j, k, t), w(i+1, j, k, t), p(i+1, j, k, t)];42
    point2 = [x(i-1, j, k, t), y(i-1, j, k, t), z(i-1, j, k, t),
u(i-1, j, k, t), v(i-1, j, k, t), w(i-1, j, k, t), p(i-1, j, k, t)];42
    point3 = [x(i, j+1, k, t), y(i, j+1, k, t), z(i, j+1, k, t),
u(i, j+1, k, t), v(i, j+1, k, t), w(i, j+1, k, t), p(i, j+1, k, t)];42
    point4 = [x(i, j-1, k, t), y(i, j-1, k, t), z(i, j-1, k, t),
u(i, j-1, k, t), v(i, j-1, k, t), w(i, j-1, k, t), p(i, j-1, k, t)];42
    point5 = [x(i, j, k-1, t), y(i, j, k-1, t), z(i, j, k-1, t),
u(i, j, k-1, t), v(i, j, k-1, t), w(i, j, k-1, t), p(i, j, k-1, t)];42
    point6 = [x(i, j, k+1, t), y(i, j, k+1, t), z(i, j, k+1, t),
u(i, j, k+1, t), v(i, j, k+1, t), w(i, j, k+1, t), p(i, j, k+1, t)];42
    point7 = [x(i, j+1, k+1, t), y(i, j+1, k+1, t), z(i, j+1,
k+1, t), u(i, j+1, k+1, t), v(i, j+1, k+1, t), w(i, j+1, k+1, t), p(i, j+1,
k+1, t)];42
    point8 = [x(i, j-1, k+1, t), y(i, j-1, k+1, t), z(i, j-1,
k+1, t), u(i, j-1, k, t), v(i, j-1, k, t), w(i, j-1, k, t), p(i, j-1, k,
t)];42
    point9 = [x(i, j+1, k-1, t), y(i, j+1, k-1, t), z(i, j+1, k-
1, t), u(i, j-1, k, t), v(i, j-1, k, t), w(i, j-1, k, t), p(i, j-1, k, t)];42
    point10 = [x(i, j-1, k-1, t), y(i, j-1, k-1, t), z(i, j-1,
k-1, t), u(i, j-1, k, t), v(i, j-1, k, t), w(i, j-1, k, t), p(i, j-1, k,
t)];42
    point11 = [x(i+1, j, k+1, t), y(i+1, j, k+1, t), z(i+1, j,
k+1, t), u(i, j-1, k, t), v(i, j-1, k, t), w(i, j-1, k, t), p(i, j-1, k,
t)];42
    point12 = [x(i-1, j, k+1, t), y(i-1, j, k+1, t), z(i-1, j,
k+1, t), u(i, j-1, k, t), v(i, j-1, k, t), w(i, j-1, k, t), p(i, j-1, k,
t)];42
    point13 = [x(i+1, j, k-1, t), y(i+1, j, k-1, t), z(i+1, j,
k-1, t), u(i, j-1, k, t), v(i, j-1, k, t), w(i, j-1, k, t), p(i, j-1, k,
t)];42
    point14 = [x(i-1, j, k-1, t), y(i-1, j, k-1, t), z(i-1, j,
k-1, t), u(i, j-1, k, t), v(i, j-1, k, t), w(i, j-1, k, t), p(i, j-1, k,
t)];42
    point15 = [x(i+1, j+1, k, t), y(i+1, j+1, k, t), z(i+1, j+1,
k, t), u(i, j-1, k, t), v(i, j-1, k, t), w(i, j-1, k, t), p(i, j-1, k, t)];42
    point16 = [x(i-1, j+1, k, t), y(i-1, j+1, k, t), z(i-1, j+1,
k, t), u(i, j-1, k, t), v(i, j-1, k, t), w(i, j-1, k, t), p(i, j-1, k, t)];42
    point17 = [x(i+1, j-1, k, t), y(i+1, j-1, k, t), z(i+1, j-1,
k, t), u(i, j-1, k, t), v(i, j-1, k, t), w(i, j-1, k, t), p(i, j-1, k, t)];42

```

```

    point18 = [x(i-1, j-1, k, t), y(i-1, j-1, k, t), z(i-1, j-1,
k, t), u(i, j-1, k, t), v(i, j-1, k, t), w(i, j-1, k, t), p(i, j-1, k, t)];42
    point19 = [x(i-1, j-1, k-1, t), y(i-1, j-1, k-1, t), z(i-1,
j-1, k-1, t), u(i, j-1, k, t), v(i, j-1, k, t), w(i, j-1, k, t), p(i, j-1,
k, t)];42
    point20 = [x(i+1, j-1, k-1, t), y(i+1, j-1, k-1, t), z(i+1,
j-1, k-1, t), u(i, j-1, k, t), v(i, j-1, k, t), w(i, j-1, k, t), p(i, j-1,
k, t)];42
    point21 = [x(i-1, j+1, k-1, t), y(i-1, j+1, k-1, t), z(i-1,
j+1, k-1, t), u(i, j-1, k, t), v(i, j-1, k, t), w(i, j-1, k, t), p(i, j-1,
k, t)];42
    point22 = [x(i-1, j-1, k+1, t), y(i-1, j-1, k+1, t), z(i-1,
j-1, k+1, t), u(i, j-1, k, t), v(i, j-1, k, t), w(i, j-1, k, t), p(i, j-1,
k, t)];42
    point23 = [x(i-1, j+1, k+1, t), y(i-1, j+1, k+1, t), z(i-1,
j+1, k+1, t), u(i, j-1, k, t), v(i, j-1, k, t), w(i, j-1, k, t), p(i, j-1,
k, t)];42
    point24 = [x(i+1, j-1, k+1, t), y(i+1, j-1, k+1, t), z(i+1,
j-1, k+1, t), u(i, j-1, k, t), v(i, j-1, k, t), w(i, j-1, k, t), p(i, j-1,
k, t)];42
    point25 = [x(i+1, j+1, k-1, t), y(i+1, j+1, k-1, t), z(i+1,
j+1, k-1, t), u(i, j-1, k, t), v(i, j-1, k, t), w(i, j-1, k, t), p(i, j-1,
k, t)];42
    point26 = [x(i+1, j+1, k+1, t), y(i+1, j+1, k+1, t), z(i+1,
j+1, k+1, t), u(i, j-1, k, t), v(i, j-1, k, t), w(i, j-1, k, t), p(i, j-1,
k, t)];42
    points = [point1; point2; point3; point4; point5; point6;
point7; point8; point9; point10;
    point11; point12; point13; point14; point15; point16;
point17; point18; point19; point20;
    point21; point22; point23; point24; point25; point26];42
    % Generate near points cloud42

42
    weights = zeros(1, 26);42

42
    for ii=1:2642
        weights(ii) = GaussianKernel(point0, points(ii), sigma);42
    end42

    u(i, j, k, t) = (u(i, j, k, t) +
weights.*points(4))/(1+sum(weights));42
    v(i, j, k, t) = (v(i, j, k, t) +
weights.*points(5))/(1+sum(weights));42
    w(i, j, k, t) = (w(i, j, k, t) +
weights.*points(6))/(1+sum(weights));42
    p(i, j, k, t) = (p(i, j, k, t) +
weights.*points(7))/(1+sum(weights));42
    end42
    %For each point, use Gaussian kernel function to smooth42
    %Velocity and pressure.42
end42
end42
42
% print data into csv files for post process and visualization42
for i = 1:size(p, 4)42
    time_slice = p(i, 1, i);42
    x_slice = x(i, 1, i);42
    y_slice = y(i, 1, i);42
    z_slice = z(i, 1, i);42
    u_slice = u(i, 1, i);42
    v_slice = v(i, 1, i);42
    w_slice = w(i, 1, i);42
    pressure_data = [];42
    row = x_slice(:);42
    col = y_slice(:);42
    depth = z_slice(:);42
    u_x = u_slice(:);42
    u_y = v_slice(:);42
    u_z = w_slice(:);42
    pre = time_slice(:);42
    pressure_data = [pressure_data; [row, col, depth, u_x, u_y, u_z, pre]];42
    pressure_data = array2table(pressure_data);42
    pressure_data.Properties.VariableNames = {'x', 'y', 'z', 'u', 'v', 'w',
'p'};42
    savfile = sprintf('data_%d.csv', i);42
    writetable(pressure_data, savfile);42
end42

```